

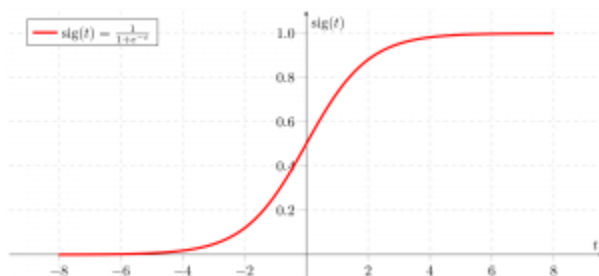
MACHINE LEARNING

UNIT-2

Logistic regression

This article discusses the basics of Logistic Regression and its implementation in Python. Logistic regression is basically a supervised classification algorithm. In a classification problem, the target variable(or output), y , can take only discrete values for given set of features(or inputs), X .

Contrary to popular belief, logistic regression IS a regression model. The model builds a regression model to predict the probability that a given data entry belongs to the category numbered as "1". Just like Linear regression assumes that the data follows a linear function, Logistic regression models the data using the sigmoid function.



Logistic regression becomes a classification technique only when a decision threshold is brought into the picture. The setting of the threshold value is a very important aspect of Logistic regression and is dependent on the classification problem itself.

The decision for the value of the threshold value is majorly affected by the values of precision and recall. Ideally, we want both precision and recall to be 1, but this seldom is the case. In case of a Precision-Recall tradeoff we use the following arguments to decide upon the threshold:-

1. **Low Precision/High Recall:** In applications where we want to reduce the number of false negatives without necessarily reducing the number false positives, we choose a decision value which has a low value of Precision or high value of Recall. For example, in a cancer diagnosis application, we do not want any affected patient to be classified as not affected without giving much heed to if the patient is being wrongfully diagnosed with cancer. This is because, the absence of cancer can be detected by further medical diseases but the presence of the disease cannot be detected in an already rejected candidate.

2. **High Precision/Low Recall:** In applications where we want to reduce the number of false positives without necessarily reducing the number false negatives, we choose a decision value which has a high value of Precision or low value of Recall. For example, if we are classifying customers whether they will react positively or negatively to a personalised advertisement, we want to be absolutely sure that the customer will react positively to the advertisement because otherwise, a negative reaction can cause a loss potential sales from the customer.

Based on the number of categories, Logistic regression can be classified as:

1. **binomial:** target variable can have only 2 possible types: "0" or "1" which may represent "win" vs "loss", "pass" vs "fail", "dead" vs "alive", etc.
2. **multinomial:** target variable can have 3 or more possible types which are not ordered(i.e. types have no quantitative significance) like "disease A" vs "disease B" vs "disease C".
3. **ordinal:** it deals with target variables with ordered categories. For example, a test score can be categorized as:"very poor", "poor", "good", "very good". Here, each category can be given a score like 0, 1, 2, 3.

Perceptron

In machine learning, the **perceptron** is an algorithm for supervised learning of binary classifiers. A binary classifier is a function which can decide whether or not an input, represented by a vector of numbers, belongs to some specific class. It is a type of linear classifier, i.e. a classification algorithm that makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

Exponential family

In probability and statistics, an **exponential family** is a parametric set of probability distributions of a certain form, specified below. This special form is chosen for mathematical convenience, based on some useful algebraic properties, as well as for generality, as exponential families are in a sense very natural sets of distributions to consider. The term **exponential class** is sometimes used in place of "exponential family", or the older term **Koopman–Darmois family**. The terms "distribution" and "family" are often used loosely: properly, *an* exponential family is a *set* of distributions, where the specific distribution varies with the parameter; however, a parametric *family* of distributions is often referred to as "a distribution" (like "the normal distribution", meaning "the family of normal distributions"), and the set of all exponential families is sometimes loosely referred to as "the" exponential family.

The concept of exponential families is credited to E. J. G. Pitman, G. Darmois, and B. O. Koopman in 1935–1936. Exponential families of distributions provides a general framework for selecting a possible alternative parameterisation of a parametric family of distributions, in terms of **natural parameters**, and for defining useful sample statistics, called the **natural sufficient statistics** of the family.

Generative learning algorithms

Generative approaches **try to build a model** of the *positives* and a model of the *negatives*. You can think of a model as a “*blueprint*” for a class. A decision boundary is formed where one model becomes more likely. As these create models of each class they can be used for generation.

To create these models, a generative learning algorithm learns the joint probability distribution $P(\mathbf{x}, \mathbf{y})$.

Now time for some maths!

The joint probability can be written as:

$$P(\mathbf{x}, \mathbf{y}) = P(\mathbf{x} | \mathbf{y}) \cdot P(\mathbf{y}) \quad \dots(i)$$

Also, using Bayes' Rule we can write:

$$P(\mathbf{y} | \mathbf{x}) = P(\mathbf{x} | \mathbf{y}) \cdot P(\mathbf{y}) / P(\mathbf{x}) \quad \dots(ii)$$

Since, to predict a class label $\mathbf{y}$, we are only interested in the arg max, the denominator can be removed from (ii).

Hence to predict the label $\mathbf{y}$ from the training example $\mathbf{x}$, generative models evaluate:

$$f(\mathbf{x}) = \operatorname{argmax}_{\mathbf{y}} P(\mathbf{y} | \mathbf{x}) = \operatorname{argmax}_{\mathbf{y}} P(\mathbf{x} | \mathbf{y}) \cdot P(\mathbf{y})$$

The most important part in the above is $P(\mathbf{x} | \mathbf{y})$. This is what allows the model to be *generative*! $P(\mathbf{x} | \mathbf{y})$ means – what $\mathbf{x}$ (*features*) are there given *class* $\mathbf{y}$. Hence, with the joint probability distribution function (i), given a $\mathbf{y}$, you can calculate (“*generate*”) its corresponding $\mathbf{x}$. **For this reason they are called generative models!**

Generative learning algorithms **make strong assumptions** on the data. To explain this let's look at a generative learning algorithm called Gaussian Discriminant Analysis (GDA)

Gaussian discriminant analysis

GDA, is a method for data classification commonly used when data can be approximated with a Normal distribution. As first step, you will need a training set, i.e. a bunch of data yet classified. These data are used to train your classifier, and obtain a discriminant function that will tell you to which class a data has higher probability to belong.

When you have your training set you need to compute the mean μ and the standard deviation σ^2 . These two variables, as you know, allow you to describe a Normal distribution.

Once you have computed the Normal distribution for each class, to classify a data you will need to compute, for each one, the probability that that data belongs to it. The class with the highest probability will be chosen as the affinity class.

More information about Discriminant Functions for the Normal Density can be found in textbook as *Pattern Classification DUDA, HART, SOTRK* or *Pattern Recognition and Machine Learning BISHOP*.

Naïve Bayes

In machine learning, **naïve Bayes classifiers** are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naïve) independence assumptions between the features. They are among the simplest Bayesian network models.

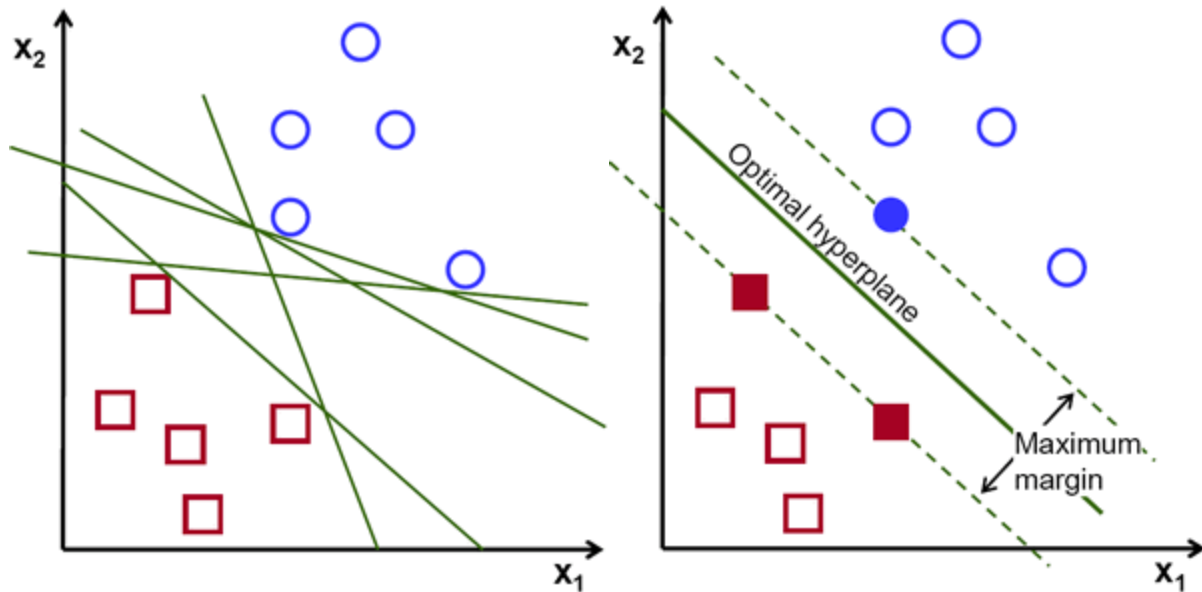
Naïve Bayes has been studied extensively since the 1960s. It was introduced (though not under that name) into the text retrieval community in the early 1960s, and remains a popular (baseline) method for text categorization, the problem of judging documents as belonging to one category or the other (document categorization)(such as spam or legitimate, sports or politics, etc.) with word frequencies as the features. With appropriate pre-processing, it is competitive in this domain with more advanced methods including support vector machines. It also finds application in automatic medical diagnosis.

Naïve Bayes classifiers are highly scalable, requiring a number of parameters linear in the number of variables (features/predictors) in a learning problem. Maximum-likelihood training can be done by evaluating a closed-form expression, which takes linear time, rather than by expensive iterative approximation as used for many other types of classifiers.

In the statistics and computer science literature, naïve Bayes models are known under a variety of names, including **simple Bayes** and **independence Bayes**. All these names reference the use of Bayes' theorem in the classifier's decision rule, but naïve Bayes is not (necessarily) a Bayesian method.

Support vector machines

The objective of the support vector machine algorithm is to find a hyperplane in an N-dimensional space(N — the number of features) that distinctly classifies the data points.

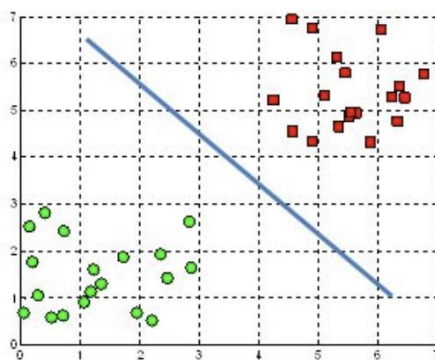


Possible hyperplanes

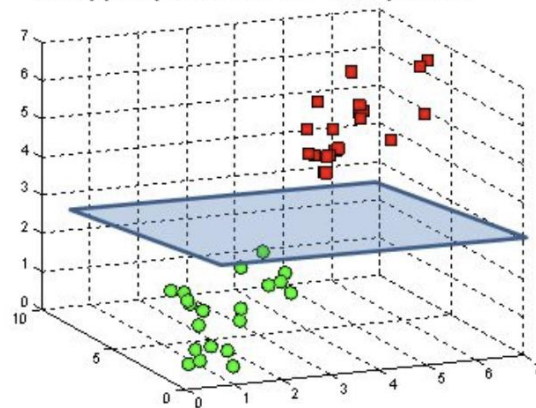
To separate the two classes of data points, there are many possible hyperplanes that could be chosen. Our objective is to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes. Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.

Hyperplanes and Support Vectors

A hyperplane in $\mathbb{R}^2$ is a line



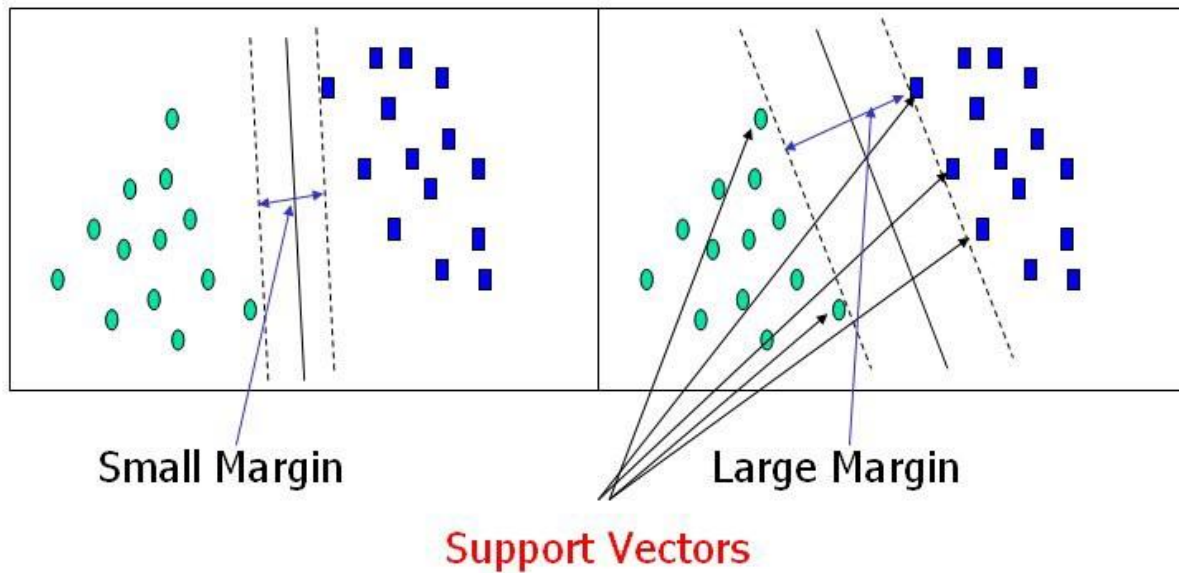
A hyperplane in $\mathbb{R}^3$ is a plane



Hyperplanes in 2D and 3D feature space

Hyperplanes are decision boundaries that help classify the data points. Data points falling on either side of the hyperplane can be attributed to different classes. Also, the

dimension of the hyperplane depends upon the number of features. If the number of input features is 2, then the hyperplane is just a line. If the number of input features is 3, then the hyperplane becomes a two-dimensional plane. It becomes difficult to imagine when the number of features exceeds 3.



Support Vectors

Support vectors are data points that are closer to the hyperplane and influence the position and orientation of the hyperplane. Using these support vectors, we maximize the margin of the classifier. Deleting the support vectors will change the position of the hyperplane. These are the points that help us build our SVM.

What is Model Selection

Given a set of models $\mathcal{M} = \{M_1, M_2, \dots, M_R\}$, choose the model that is **expected to do the best on the test data**. $\mathcal{M}$ may consist of:

- Same learning model with **different complexities** or **hyperparameters**
 - Nonlinear Regression: Polynomials with different degrees
 - K -Nearest Neighbors: Different choices of K
 - Decision Trees: Different choices of the number of levels/leaves
 - SVM: Different choices of the misclassification penalty hyperparameter C
 - Regularized Models: Different choices of the regularization parameter
 - Kernel based Methods: Different choices of kernels
 - .. and almost any learning problem
- Different learning models (e.g., SVM, KNN, DT, etc.)

Note: Usually considered in supervised learning contexts but unsupervised learning too faces this issue (e.g., “how many clusters” when doing clustering)

Feature Selection

Selecting a useful subset from all the features

Why Feature Selection?

- Some algorithms scale (computationally) poorly with increased dimension
- Irrelevant features can confuse some algorithms
- Redundant features adversely affect regularization
- Removal of features can increase (relative) margin (and generalization)
- Reduces data set and resulting model size
- Note: **Feature Selection** is different from **Feature Extraction**
 - The latter transforms original features to get a small set of new features
 - More on feature extraction when we cover **Dimensionality Reduction**

Combining Classifiers

Vote provides a baseline method for combining classifiers. The default scheme is to average their probability estimates or numeric predictions, for classification and regression, respectively. Other combination schemes are available—for example, using majority voting for classification. MultiScheme selects the best classifier from a set of candidates using cross-validation of percentage accuracy or mean-squared error for classification and regression, respectively. The number of folds is a parameter. Performance on training data can be used instead.

Stacking combines classifiers using stacking (see Section 8.7, page 369) for both classification and regression problems. You specify the base classifiers, the metalearner, and the number of cross-validation folds. StackingC implements a more efficient variant for which the metalearner must be a numeric prediction scheme (Seewald, 2002). In Grading, the inputs to the metalearner are base-level predictions that have been marked (i.e., “graded”) as correct or incorrect. For each base classifier, a metalearner is learned that predicts when the base classifier will err. Just as stacking may be viewed as a generalization of voting, grading generalizes selection by cross-validation (Seewald and Fürnkranz, 2001).

Bagging

Bootstrap aggregating, also called **bagging** (from bootstrap aggregating), is a machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in statistical classification and regression. It also reduces variance and helps to avoid overfitting.

Boosting

The term '**Boosting**' refers to a family of algorithms which converts weak learner to strong learners. **Boosting** is an ensemble method for improving the model predictions of any given learning algorithm. The idea of **boosting** is to train weak learners sequentially, each trying to correct its predecessor.

Evaluating and debugging learning algorithms

Once you have defined your problem and prepared your data you need to apply machine learning algorithms to the data in order to solve your problem.

You can spend a lot of time choosing, running and tuning algorithms. You want to make sure you are using your time effectively to get closer to your goal.

In this post you will step through a process to rapidly test algorithms and discover whether or not there is structure in your problem for the algorithms to learn and which algorithms are effective.

Test Harness

You need to define a test harness. The test harness is the data you will train and test an algorithm against and the performance measure you will use to assess its performance. It is important to define your test harness well so that you can focus on evaluating different algorithms and thinking deeply about the problem.

The goal of the test harness is to be able to quickly and consistently test algorithms against a fair representation of the problem being solved. The outcome of testing multiple algorithms against the harness will be an estimation of how a variety of algorithms perform on the problem against a chosen performance measure. You will know which algorithms might be worth tuning on the problem and which should not be considered further.

The results will also give you an indication of how learnable the problem is. If a variety of different learning algorithms universally perform poorly on the problem, it may be an indication of a lack of structure available to algorithms to learn. This may be because there actually is a lack of learnable structure in the selected data or it may be an opportunity to try different transforms to expose the structure to the learning algorithms.

Performance Measure

The performance measure is the way you want to evaluate a solution to the problem. It is the measurement you will make of the predictions made by a trained model on the test dataset.

Performance measures are typically specialized to the class of problem you are working with, for example classification, regression, and clustering. Many standard performance measures will give you a score that is meaningful to your problem domain. For example, classification accuracy for classification (total correct correction divided by the total predictions made multiple by 100 to turn it into a percentage).

You may also want a more detailed breakdown of performance, for example, you may want to know about the false positives on a spam classification problem because good email will be marked as spam and cannot be read.

There are many standard performance measures to choose from. You rarely have to devise a new performance measure yourself as you can generally find or adapt one that best captures the requirements of the problem being solved. Look to similar problems you uncovered and at the performance measures used to see if any can be adopted.

Test and Train Datasets

From the transformed data, you will need to select a test set and a training set. An algorithm will be trained on the training dataset and will be evaluated against the test set. This may be as simple as selecting a random split of data (66% for training, 34% for testing) or may involve more complicated sampling methods.

A trained model is not exposed to the test dataset during training and any predictions made on that dataset are designed to be indicative of the performance of the model in general. As such you want to make sure the selection of your datasets are representative of the problem you are solving.

Cross Validation

A more sophisticated approach than using a test and train dataset is to use the entire transformed dataset to train and test a given algorithm. A method you could use in your test harness that does this is called cross validation.

It first involves separating the dataset into a number of equally sized groups of instances (called folds). The model is then trained on all folds exception one that was left out and the prepared model is tested on that left out fold. The process is repeated so that each fold get's an opportunity at being left out and acting as the test dataset. Finally, the performance measures are averaged across all folds to estimate the capability of the algorithm on the problem.

For example, a 3-fold cross validation would involve training and testing a model 3 times:

- #1: Train on folds 1+2, test on fold 3
- #2: Train on folds 1+3, test on fold 2
- #3: Train on folds 2+3, test on fold 1

The number of folds can vary based on the size of your dataset, but common numbers are 3, 5, 7 and 10 folds.

The goal is to have a good balance between the size and representation of data in your train and test sets.

When you're just getting started, stick with a simple split of train and test data (such as 66%/34%) and move onto cross validation once you have more confidence.

Testing Algorithms

When starting with a problem and having defined a test harness you are happy with, it is time to spot check a variety of machine learning algorithms. Spot checking is useful because it allows you to very quickly see if there is any learnable structures in the data and estimate which algorithms may be effective on the problem.

Spot checking also helps you work out any issues in your test harness and make sure the chosen performance measure is appropriate.

The best first algorithm to spot check is a random. Plug in a random number generator to generate predictions in the appropriate range. This should be the worst “algorithm result” you achieve and will be the measure by which all improvements can be assessed.

Select 5-10 standard algorithms that are appropriate for your problem and run them through your test harness. By standard algorithms, I mean popular methods no special configurations. Appropriate for your problem means that the algorithms can handle regression if you have a regression problem.

Choose methods from the groupings of algorithms we have already reviewed. I like to include a diverse mix and have 10-20 different algorithms drawn from a diverse range of algorithm types. Depending on the library I am using, I may spot check up to a 50+ popular methods to flush out promising methods quickly.

If you want to run a lot of methods, you may have to revisit data preparation and reduce the size of your selected dataset. This may reduce your confidence in the results, so test with various data set sizes. You may like to use a smaller size dataset for algorithm spot checking and a fuller dataset for algorithm tuning.

Classification errors

There are multiple types of *errors* associated with machine learning and predictive analytics. The primary types are *in-sample* and *out-of-sample* errors. In-sample errors (aka *re-substitution errors*) are the error rate found from the training data, i.e., the data used to build predictive models.

Out-of-sample errors (aka *generalisation errors*) are the error rates found on a new data set, and are the most important since they represent the potential performance of a given predictive model on new and unseen data.

In-sample error rates may be very low and seem to be indicative of a high-performing model, but one must be careful, as this may be due to overfitting as mentioned, which would result in a model that is unable to generalise well to new data.

Training and validation data is used to build, validate, and tune a model, but test data is used to evaluate model performance and generalisation capability. One very important point to note is that prediction performance and error analysis should only be done on test data, when evaluating a model for use on non-training or new data (out-of-sample).

Generally speaking, model performance on training data tends to be optimistic, and therefore data errors will be less than those involving test data. There are tradeoffs between the types of errors that a machine learning practitioner must consider and often choose to accept.

For binary classification problems, there are two primary types of errors. *Type 1* errors (false positives) and *Type 2* errors (false negatives). It's often possible through model selection and tuning to increase one while decreasing the other, and often one must choose which error type is more acceptable. This can be a major tradeoff consideration depending on the situation.

A typical example of this tradeoff dilemma involves cancer diagnosis, where the positive diagnosis of having cancer is based on some test. In this case, a false positive means that someone is told that have have cancer when they do not. Conversely, the false negative case is when someone is told that they do not have cancer when they actually do. If no model is perfect, then in the example above, which is the more acceptable error type? In other words, of which one can we accept to a greater degree?

Telling someone they have cancer when they don't can result in tremendous emotional distress, stress, additional tests and medical costs, and so on. On the other hand, failing to detect cancer in someone that actually has it can mean the difference between life and death.

In the spam or ham case, neither error type is nearly as serious as the cancer case, but typically email vendors err slightly more on the side of letting some spam get into your inbox as opposed to you missing a very important email because the spam classifier is too aggressive.